



//Veera Manual Audit Report

Prepared For: Akshay Khare | Sam Noble
Thursday, January 15, 2026

About Octane

Founded in 2023 and headquartered in San Francisco, Octane delivers a truly holistic security solution through continuous code reviews that protect protocols throughout their entire development lifecycle. We combine software-first vulnerability analysis with world-class manual security talent, enabling teams to ship faster while maintaining the highest security standards. Our clients including Circle (USDC), Avalanche, Monad, and Usual trust us to protect \$186.6B+ in assets because we catch vulnerabilities that traditional assessments miss.

Clients choose Octane for:

- Our software-first approach: We've built a platform trained on thousands of real-world vulnerabilities that consistently identifies complex exploit patterns traditional methods miss. Every commit and pull request receives automatic review through native CI/CD integration, with vulnerabilities identified through deep dependency analysis and exploit path tracing.
- Our novel research: Octane's security researchers who've collectively earned over \$1M in bug bounties provide comprehensive manual reviews on an as-needed basis, maximizing code security and developing further insights for automated protection.
- Our proven track record preventing exploits: We've discovered 100+ vulnerabilities across 86M+ lines of code that were missed in traditional security reviews. We prevented exploits similar to Rikkei Finance (\$1.1M), caught complex slippage bugs in Uniswap integrations, identified oracle mispricing vulnerabilities, and found assembly errors that evaded static analyzers. Our platform adapts to your team's preferences, dramatically reducing false positives as the platform learns your codebase while providing exact code modifications and exploit scenarios for immediate remediation.
- Our commitment to ecosystem security: We partner directly with blockchain foundations to establish network-wide security baselines. Integration requires no code changes and takes minutes to deploy, after which teams receive continuous protection with rapid response. We offer flexible engagement from basic reviews to enterprise coverage with 24/7 support and formal security reports, supporting all EVM-compatible chains with developing Rust capabilities.

Through our holistic approach, we transform security from a development bottleneck into an enabler of velocity and confidence. To learn how continuous security accelerates development while protecting users, visit [octane.security](https://www.octane.security).

DISCLAIMER

A smart contract security review can never verify the complete absence of vulnerabilities. This is a time, resource and expertise bound effort where we try to find as many vulnerabilities as possible. We also rely on information that is provided by the client, its affiliates and partners for vulnerability identification. We can not guarantee the absence of vulnerabilities, issues, flaws, or defects after the review. It is up to the client to decide whether to implement recommendations and we take no liability for invalid recommendations. Subsequent security reviews, bug bounty programs and on-chain monitoring are strongly recommended.

Security Assessment Summary

The scope of this review was limited to files at commit:

[d03eb7f6c1450bcd36cc1e354bf3d6e9f60dff60](#)

The fix is at commit [1f345e0e7887decad23abef74c61b991c38079d4](#)

Scope

The following smart contracts were in scope of the audit:

- src/Veera.sol
 - script/DeployVeera.s.sol
-

Findings

Finding Number	Finding Title
I-1	Missing Explicit Inheritance From ERC20 Base Contract (FIXED)
I-2	Ownership Renouncement Can Permanently Disable Pause Control (FIXED)

I-1 Missing Explicit Inheritance From ERC20 Base Contract

Description

The contract inherits from `ERC20Burnable`, `ERC20Pausable`, `Ownable`, and `ERC20Permit`. Although `ERC20` is imported, it is not explicitly listed in the inheritance chain:

```
import {ERC20} from "@openzeppelin/contracts/token/ERC20/ERC20.sol";
import {ERC20Burnable} from
"@openzeppelin/contracts/token/ERC20/extensions/ERC20Burnable.sol";
import {ERC20Pausable} from
"@openzeppelin/contracts/token/ERC20/extensions/ERC20Pausable.sol";
import {ERC20Permit} from
"@openzeppelin/contracts/token/ERC20/extensions/ERC20Permit.sol";
import {Ownable} from "@openzeppelin/contracts/access/Ownable.sol";

contract Veera is ERC20Burnable, ERC20Pausable, Ownable, ERC20Permit {
    // ...
}
```

While this works because the inherited extensions already depend on `ERC20`, omitting it from the inheritance list can reduce clarity and make the contract structure less explicit.

Recommendation

Explicitly inherit from `ERC20` to improve readability and consistency, in line with the OpenZeppelin Wizard recommendations:

```
contract Veera is ERC20, ERC20Burnable, ERC20Pausable, Ownable, ERC20Permit
{
    // ...
}
```

This makes the contract's inheritance hierarchy clearer and easier to maintain.

Resolution

This issue is fixed by explicitly including ERC20.

I-2 Ownership Renouncement Can Permanently Disable Pause Control

Description

The token includes a pausability mechanism that can only be triggered by the owner, which is useful in emergency situations:

```
function pause() external onlyOwner {  
    _pause();  
}
```

```
function unpause() external onlyOwner {  
    _unpause();  
}
```

However, the owner can renounce ownership by calling `renounceOwnership`, which transfers ownership to the zero address:

```
function renounceOwnership() public virtual onlyOwner {  
    _transferOwnership(address(0));  
}
```

After ownership is renounced, there is no longer an owner, meaning the pause and unpause functionality becomes permanently unavailable.

In an edge case, if the owner pauses the token and then accidentally renounces ownership, the token will remain paused forever, with no way to recover normal operation.

Recommendation

To avoid this risk, consider one of the following approaches:

- Disable ownership renouncement by overriding `renounceOwnership` and reverting in its implementation.
- Clearly document that the owner must never renounce ownership.
- Adopt a role-based access control pattern where critical responsibilities (e.g., minting, pausing, and administration) are assigned to separate addresses instead of a single owner.

Resolution

This issue is fixed by introducing Roles to separate minting, pausing and admin control.